Universidad Católica de Córdoba

“PROYECTO INTEGRADOR: ANALIZADOR SINTÁCTICO PARA MINIHTML”

Integrantes: ARTIGAU, Delfina, GRILLO, Facundo, GATTI, Pedro y LASA, Maria Emilia.

Materia: *“Lenguajes Formales y Autómatas”*.

Docente/s: *Lorena Bergamini*

Cátedra: A / A1

Carrera: Ingeniería en Informática

Fecha de Entrega: 03 / 11 / 2026

1. Introducción

Este informe presenta el diseño formal de un analizador sintáctico para MiniHTML, un subconjunto reducido de HTML en el que solo interesa la estructura de las etiquetas. El trabajo se organizó en tres etapas: la especificación de una gramática libre de contexto que genera el lenguaje, el diseño de un autómata a pila que lo reconoce y la implementación en Python de un simulador de ese autómata.

Las cadenas de MiniHTML se forman con siete símbolos. Cada etiqueta se considera un único símbolo (token) y todo fragmento de texto plano se representa con el token genérico TEXTO: Σ = { , , , , , , TEXTO }.

Una cadena es válida cuando cada etiqueta abierta se cierra con la etiqueta del mismo tipo y los cierres respetan el orden inverso al de apertura: la última etiqueta abierta es la primera que debe cerrarse (orden LIFO, last in, first out). El documento vacío y el texto sin etiquetas también son válidos. Por ejemplo, TEXTO TEXTO   es válida, mientras que   TEXTO   no lo es, porque cierra el párrafo antes que la negrita.

2. Por qué un autómata finito no alcanza

Un autómata finito tiene una cantidad fija de estados y ninguna otra memoria: todo lo que conserva de la parte ya leída de la cadena está codificado en el estado en que se encuentra. Eso alcanza para propiedades acotadas, como la paridad de una cantidad de símbolos, pero no para MiniHTML. En este lenguaje la profundidad de anidamiento no tiene límite, y para decidir si los cierres son correctos hay que recordar cuántas etiquetas están abiertas y en qué orden se abrieron.

Intuitivamente, si el autómata tiene k estados y lee una cadena que abre más de k párrafos seguidos, por el principio del palomar hay dos cantidades distintas de párrafos abiertos que lo dejan en el mismo estado. Desde ese estado no puede saber cuántos cierres faltan, así que termina aceptando alguna cadena desbalanceada o rechazando alguna correcta.

Formalmente, lo demostramos con el lema de bombeo para lenguajes regulares. Supongamos que MiniHTML es regular y sea n su constante de bombeo. Tomamos la cadena $\omega = \texttt{<p>}^n$ TEXTO $\texttt{</p>}^n$, que pertenece al lenguaje y tiene longitud $2n + 1 \geq n$. Cualquier descomposición $\omega = \alpha\beta\gamma$ con $|\alpha\beta| \leq n$ y $\beta \neq \lambda$ deja a α y β dentro de los primeros n símbolos, que son todos . Entonces $\beta = \texttt{<p>}^s$ con $s \geq 1$. Al bombear con i = 0 se obtiene $\alpha\gamma = \texttt{<p>}^{n-s}$ TEXTO $\texttt{</p>}^n$, que tiene más cierres que aperturas y por lo tanto no pertenece al lenguaje. Esto contradice el lema, de modo que MiniHTML no es regular. Es el mismo argumento que prueba que no son regulares $a^n b^n$ ni el lenguaje de paréntesis balanceados.

Se necesita entonces un modelo con memoria no acotada. Como las etiquetas se cierran en orden inverso al de apertura, la estructura adecuada es una pila: al abrir una etiqueta se apila un símbolo que la representa y al cerrarla se verifica que ese símbolo esté en el tope y se lo desapila. Esto ubica a MiniHTML entre los lenguajes independientes del contexto (tipo 2 de la jerarquía de Chomsky), que son los generados por gramáticas libres de contexto y los reconocidos por autómatas a pila.

3. Fase 1: Gramática libre de contexto

Definición formal

Definimos la gramática G = (V, Σ, S, P), donde:

- V = { S, E } es el conjunto de variables o no terminales.
- Σ = { , , , , , , TEXTO } es el conjunto de terminales.
- S es el símbolo inicial.
- P es el conjunto de producciones de la tabla siguiente.

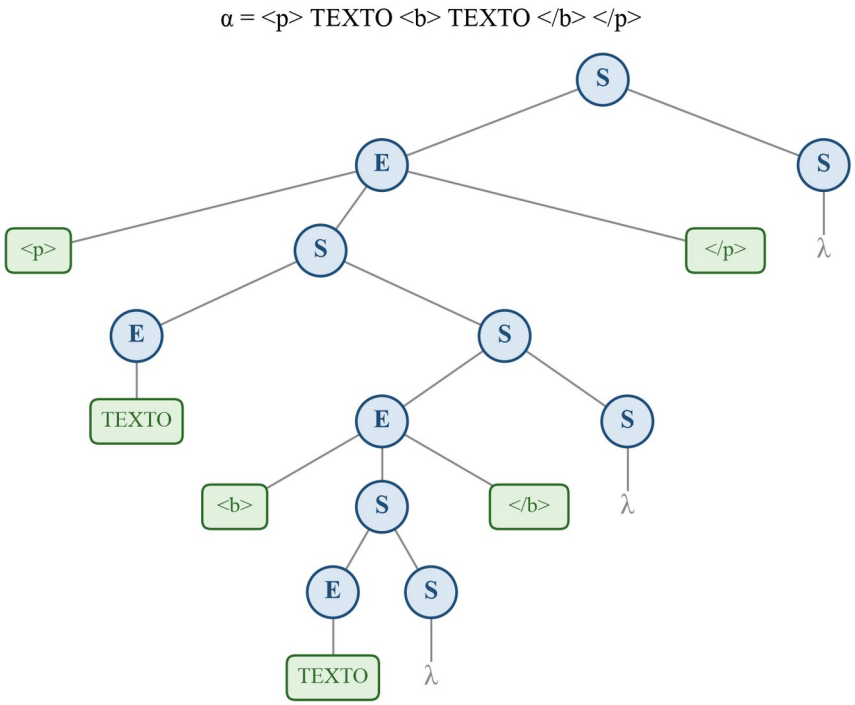
La variable S representa una secuencia de cero o más elementos ubicados en el mismo nivel; describe tanto el documento completo como el contenido de cualquier etiqueta. La variable E representa un único elemento: un texto o un bloque delimitado por una etiqueta de apertura y su cierre.

N°	Producción	Significado
1	S → E S	Una secuencia es un elemento seguido de otra secuencia.
2	S → λ	Una secuencia puede ser vacía.
3	E → TEXTO	Un elemento puede ser texto plano.
4	E → S	Bloque en negrita con contenido S.
5	E → S	Bloque en cursiva con contenido S.
6	E → S	Párrafo con contenido S.

Justificación del diseño

- **Orden LIFO.** En las producciones 4 a 6, la apertura y el cierre de una etiqueta se generan en el mismo paso, con la variable S entre ambos. Todo lo que se derive de esa S queda encerrado entre los dos símbolos, por lo que cualquier etiqueta interna se completa antes de que aparezca el cierre exterior. No hay forma de derivar un cruce como   .
- **Elementos consecutivos.** La recursión por derecha S → E S, junto con S → λ, permite concatenar cualquier cantidad de elementos en un mismo nivel, por ejemplo, un párrafo seguido de una cursiva.
- **Cadena vacía y etiquetas vacías.** Gracias a S → λ, el documento vacío y bloques como   pertenecen al lenguaje.
- **Ausencia de ambigüedad.** En una derivación más a la izquierda, el próximo símbolo de la cadena determina la producción a aplicar. Si es TEXTO o una apertura, S se reemplaza por E S, y E por la producción que corresponde a ese símbolo; si es un cierre o la cadena terminó, S se reemplaza por λ. Como en cada paso hay una sola opción, cada cadena tiene un único árbol de derivación. Separar S de E es lo que lo garantiza: una gramática con una regla como S → S S sería ambigua, porque tres elementos seguidos podrían agruparse de dos maneras.

***Árbol de derivación de α = TEXTO TEXTO ***

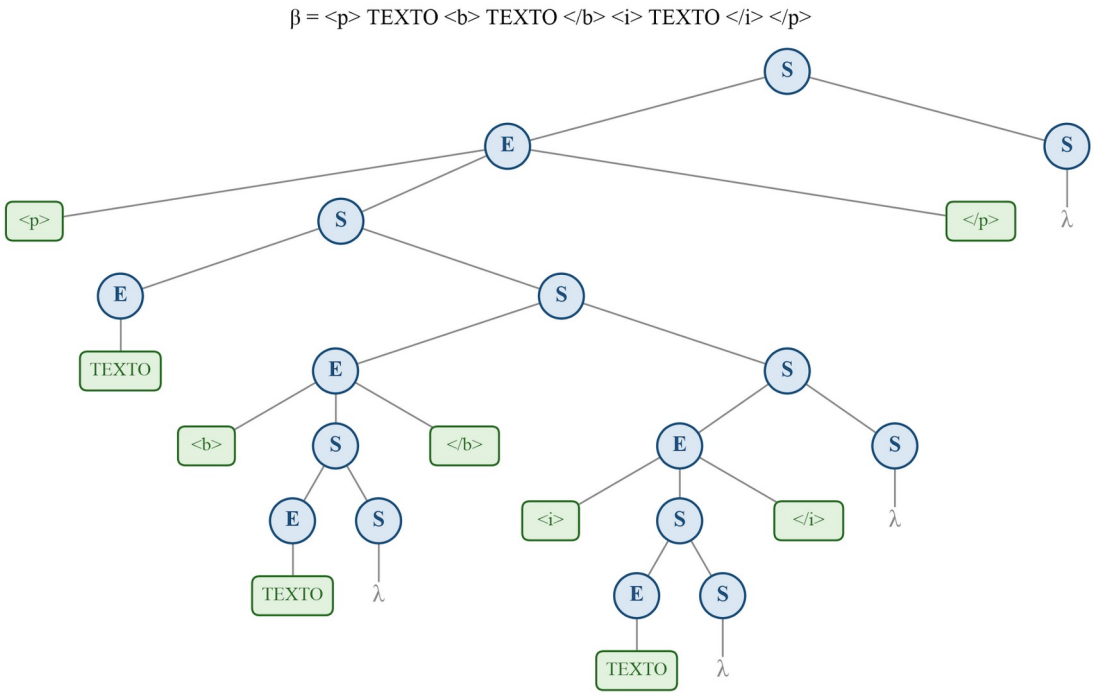


La raíz del árbol es el símbolo inicial S, los nodos internos son variables y las hojas son terminales o λ. Leyendo las hojas de izquierda a derecha, sin las λ, se obtiene la cadena.

Derivación más a la izquierda (entre paréntesis, el número de la producción aplicada):

S ⇒ E S	(1)
⇒ S S	(6)
⇒ E S S	(1)
⇒ TEXTO S S	(3)
⇒ TEXTO E S S	(1)
⇒ TEXTO S S S	(4)
⇒ TEXTO E S S S	(1)
⇒ TEXTO TEXTO S S S	(3)
⇒ TEXTO TEXTO S S	(2)
⇒ TEXTO TEXTO S	(2)
⇒ TEXTO TEXTO	(2)

***Árbol de derivación de β = TEXTO TEXTO   TEXTO ***



Derivación más a la izquierda:

S ⇒ E S	(1)
⇒ S S	(6)
⇒ E S S	(1)
⇒ TEXTO S S	(3)
⇒ TEXTO E S S	(1)
⇒ TEXTO S S S	(4)
⇒ TEXTO E S S S	(1)
⇒ TEXTO TEXTO S S S	(3)
⇒ TEXTO TEXTO S S	(2)
⇒ TEXTO TEXTO E S S	(1)
⇒ TEXTO TEXTO S S S	(5)
⇒ TEXTO TEXTO E S S S	(1)
⇒ TEXTO TEXTO TEXTO S S S	(3)
⇒ TEXTO TEXTO TEXTO S S	(2)
⇒ TEXTO TEXTO TEXTO S	(2)
⇒ TEXTO TEXTO TEXTO	(2)

4. Fase 2: Autómata a pila

Modelo elegido

Diseñamos un autómata finito con pila no determinista con transiciones λ (AFPN-λ) que acepta por estado final. Cada etiqueta de apertura apila un símbolo que la identifica, el texto no modifica la pila y cada etiqueta de cierre desapila el símbolo del tope solo si corresponde a la misma etiqueta. Cuando la entrada se termina de leer y en la pila queda únicamente el marcador de fondo, una transición λ lleva al estado de aceptación.

El autómata es no determinista por un único motivo: en el estado q0 y con Z0 en el tope hay dos opciones, leer el próximo símbolo o tomar la transición λ hacia qf. Esto no produce aceptaciones incorrectas, porque qf no tiene transiciones salientes: una rama que pasa a qf antes de consumir toda la entrada queda detenida y no acepta. La única rama que puede aceptar es la que toma la transición λ al final de la cadena.

Definición formal

Definimos M = (Q, Σ, Γ, δ, q0, Z0, F), donde:

- Q = { q0, qf } es el conjunto de estados.
- Σ = { , , , , , , TEXTO } es el alfabeto de entrada.
- Γ = { Z0, B, I, P } es el alfabeto de la pila. B, I y P indican que hay abierta una etiqueta , o , respectivamente.
- δ: Q × (Σ ∪ {λ}) × Γ → P(Q × Γ*) es la función de transición, detallada a continuación.
- q0 es el estado inicial, en el que se procesa toda la entrada.
- Z0 es el símbolo inicial de la pila y marca su fondo.
- F = { qf } es el conjunto de estados de aceptación.

Función de transición

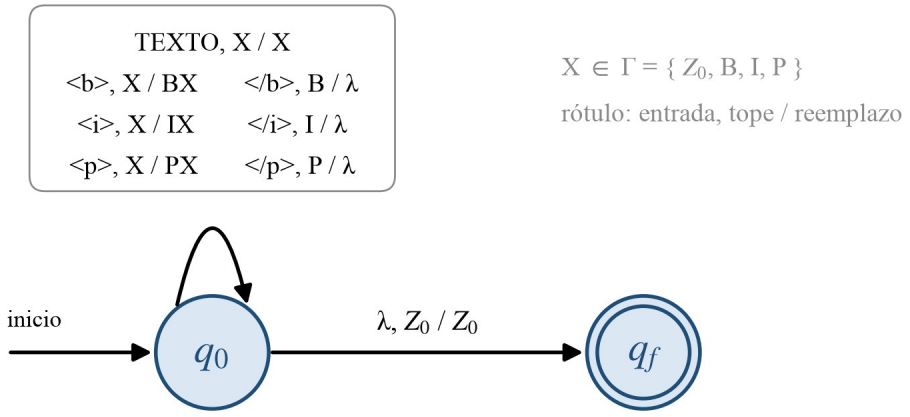
Escribimos δ(q, a, z) = (q', γ) para indicar que, en el estado q, leyendo a (o sin leer, si a = λ) y con z en el tope de la pila, el autómata pasa al estado q' y reemplaza z por la cadena γ. Si γ = Yz se apila Y; si γ = z la pila no cambia; si γ = λ se desapila z. Como cada δ(q, a, z) tiene a lo sumo un elemento, (q', γ) abrevia el conjunto {(q', γ)}. En la tabla, X representa cualquier símbolo de Γ.

N°	Estado	Entrada	Tope	δ(q, a, z)	Pila
1	q0	TEXTO	X	(q0, X)	Sin cambios
2	q0		X	(q0, BX)	Apila B
3	q0		X	(q0, IX)	Apila I
4	q0		X	(q0, PX)	Apila P
5	q0		B	(q0, λ)	Desapila B
6	q0		I	(q0, λ)	Desapila I
7	q0		P	(q0, λ)	Desapila P
8	q0	λ	Z0	(qf, Z0)	Acepta

Para cualquier combinación que no figura en la tabla, δ(q, a, z) = ∅ y el autómata se detiene sin aceptar. Así se rechazan los tres errores posibles: cerrar una etiqueta distinta de la que está en el tope (por ejemplo, leer con B en el tope), cerrar una etiqueta cuando no hay ninguna abierta (leer un cierre con Z0 en el tope) y terminar la entrada con etiquetas abiertas (no se puede pasar a qf porque el tope no es Z0).

Diagrama de transiciones

Cada arco se rotula como «entrada, tope / reemplazo». Todas las lecturas son lazos sobre q0; la única transición entre estados distintos es la transición λ hacia qf.



Trazas de configuraciones instantáneas

Una configuración instantánea es la terna (q, ω, γ): el estado actual, la parte de la entrada que falta leer y el contenido de la pila, con el tope a la izquierda. El símbolo ⊢ indica un paso de cómputo y a la derecha figura el número de la transición aplicada.

Cadena α = TEXTO TEXTO

(q0, TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , B P Z0)	(2)
⊢ (q0, , B P Z0)	(1)
⊢ (q0, , P Z0)	(5)
⊢ (q0, λ, Z0)	(7)
⊢ (qf, λ, Z0)	(8)

La entrada se consumió completa y el autómata terminó en qf ∈ F, por lo tanto α ∈ L(M).

Cadena β = TEXTO TEXTO   TEXTO

(q0, TEXTO TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO TEXTO , P Z0)	(1)
⊢ (q0, TEXTO TEXTO , B P Z0)	(2)
⊢ (q0, TEXTO , B P Z0)	(1)
⊢ (q0, TEXTO , P Z0)	(5)
⊢ (q0, TEXTO , I P Z0)	(3)
⊢ (q0, , I P Z0)	(1)
⊢ (q0, , P Z0)	(6)
⊢ (q0, λ, Z0)	(7)
⊢ (qf, λ, Z0)	(8)

β ∈ L(M).

Cadena γ = TEXTO TEXTO   TEXTO

(q0, TEXTO TEXTO TEXTO , Z0)	inicial
⊢ (q0, TEXTO TEXTO TEXTO , P Z0)	(4)
⊢ (q0, TEXTO TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , P Z0)	(1)
⊢ (q0, TEXTO , Z0)	(7)
⊢ (q0, TEXTO , I Z0)	(3)
⊢ (q0, , I Z0)	(1)
⊢ (q0, λ, Z0)	(6)
⊢ (qf, λ, Z0)	(8)

γ ∈ L(M). En esta cadena el párrafo se cierra y la pila vuelve a Z0 antes de abrir la cursiva, lo que muestra el manejo de elementos consecutivos en el nivel superior.

***Ejemplo de rechazo: TEXTO ***

(q0, TEXTO , Z0)	inicial
⊢ (q0, TEXTO , P Z0)	(4)
⊢ (q0, TEXTO , B P Z0)	(2)
⊢ (q0, , B P Z0)	(1)
Bloqueo: δ(q0, , B) = ∅	rechaza

Al leer el tope es B, es decir, la última etiqueta abierta es . No hay transición definida para ese caso, el autómata se detiene y la cadena no pertenece a L(M).

Correspondencia entre G y M

Los dos modelos describen el mismo lenguaje, como asegura la equivalencia entre gramáticas libres de contexto y autómatas a pila. La relación entre sus elementos es directa:

Gramática G	Autómata M
E → TEXTO	Transición 1: se lee TEXTO sin modificar la pila.
Apertura en E → <x> S </x>	Se apila el símbolo de la etiqueta (transiciones 2 a 4).
Cierre en E → <x> S </x>	Se desapila si el tope coincide (transiciones 5 a 7).
Variable S dentro de una etiqueta (anidamiento)	La pila crece en profundidad.
S → E S (elementos consecutivos)	Lecturas sucesivas en q0 con la pila a la misma altura.
S → λ	Se vuelve al símbolo de la etiqueta contenedora o a Z0.
λ ∈ L(G)	(q0, λ, Z0) ⊢ (qf, λ, Z0)

5. Fase 3: Implementación

El reconocedor está implementado en Python 3, sin dependencias externas, en el archivo src/parser.py del repositorio. De las dos estrategias posibles elegimos la opción B: simular el autómata M, porque cada rama del código corresponde a filas de la tabla de transiciones y la pila queda a la vista. La gramática G no está programada; su relación con el código es la que muestra la correspondencia entre G y M.

El programa se organiza en cuatro partes:

- **Definición de las etiquetas.** El diccionario ETIQUETAS asocia cada etiqueta con su símbolo de pila, y a partir de él se construyen APERTURAS y CIERRES. Por eso, incorporar una etiqueta nueva requiere agregar una sola entrada.
- **Analizador léxico.** La función tokenizar convierte la entrada en la lista de tokens de Σ. Acepta tanto la notación con TEXTO como HTML real, donde cada palabra se convierte en un token TEXTO, y conserva las etiquetas mal formadas para que se rechacen como error léxico.
- **Simulador del autómata M.** La función simular usa una lista de Python como pila, inicializada con Z0, y recorre los tokens una sola vez aplicando las transiciones de la tabla. También registra la traza de configuraciones instantáneas, que arma la función configuracion.
- **Interfaz.** La función analizar_cadena muestra los tokens, la traza y el resultado. El programa recibe la cadena como argumento o, sin argumentos, abre un modo interactivo.

Cada transición de M tiene su lugar en el código:

Transición de M	Código en simular
1: leer TEXTO	if token == "TEXTO": no se modifica la pila.
2 a 4: leer una apertura	pila.append(APERTURAS[token])
5 a 7: leer un cierre que coincide	tope == CIERRES[token] y pila.pop()
8: fin de la entrada con Z0 en el tope	Si pila == ["Z0"], se agrega la configuración en qf y se acepta.
$\delta(q, a, z) = \varnothing$	return False, mensaje, traza

El simulador no necesita explorar ramas: como la única que puede aceptar es la que toma la transición 8 con la entrada ya consumida, el programa la aplica solo al terminar de leer.

Ante una cadena inválida, el programa informa el motivo del rechazo:

Error	Ejemplo	Mensaje del programa
Anidamiento cruzado	TEXTO	Error de anidamiento: se leyó ' ' con B en el tope.
Cierre sin apertura	TEXTO	Error sintáctico: se cerró '' sin apertura previa.
Etiquetas sin cerrar	TEXTO	Error sintáctico: etiquetas sin cerrar: .
Etiqueta desconocida	<u> TEXTO </u>	Error léxico: '<u>' no pertenece a Σ.
Etiqueta mal formada	TEXTO <b	Error léxico: '<b' no pertenece a Σ.

El archivo tests/test_parser.py contiene 12 pruebas automatizadas, que incluyen α, β y γ, la cadena vacía y los tipos de error de esta tabla.

6. Conclusiones

MiniHTML exige comprobar que las etiquetas se cierren en orden inverso al de apertura y sin límite de profundidad, algo que un autómata finito no puede hacer porque su memoria es acotada, como muestra el lema de bombeo. La gramática G describe el lenguaje generando cada apertura junto con su cierre, y el autómata a pila M lo reconoce guardando en la pila las etiquetas abiertas. El programa simula M y clasificó correctamente todas las cadenas de prueba, válidas e inválidas; la equivalencia entre gramáticas libres de contexto y autómatas a pila respalda que G y M describen el mismo lenguaje.